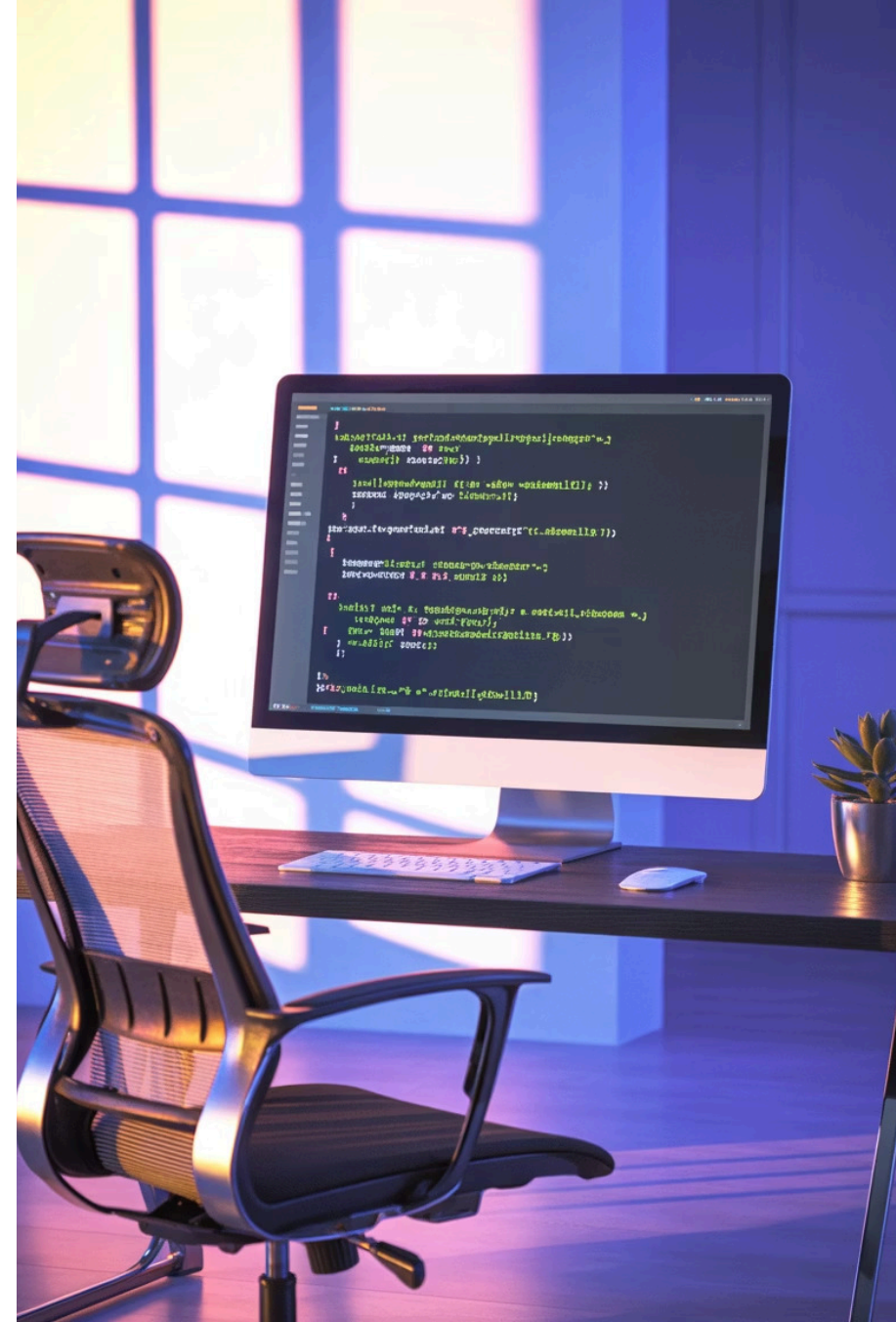


Programación Orientada a Objetos en JavaScript

Domina los conceptos fundamentales de POO para crear aplicaciones web más organizadas, reutilizables y mantenibles. Una guía práctica con ejemplos reales para desarrolladores en formación.



¿Qué son los Objetos en JavaScript?

Los objetos son la base de JavaScript y representan entidades del mundo real mediante propiedades (datos) y métodos (acciones). Son colecciones estructuradas que nos permiten organizar información relacionada de forma lógica.

Sintaxis Básica

```
let persona = {  
  nombre: "Ana",  
  edad: 25,  
  saludar: function() {  
    console.log("Hola, soy " + this.nombre);  
  }  
};
```

```
console.log(persona.nombre); // Ana  
persona.saludar(); // Hola, soy Ana
```



Los objetos encapsulan datos y comportamiento, facilitando la organización del código y mejorando la legibilidad. Cada propiedad tiene un nombre (clave) y un valor asociado.

Clases: Plantillas para Crear Objetos

Las clases son plantillas reutilizables que definen la estructura y comportamiento de los objetos. Introducidas en ES6, proporcionan una sintaxis más clara y familiar para trabajar con programación orientada a objetos.

Constructor

Método especial que se ejecuta automáticamente al crear una nueva instancia de la clase

Métodos

Funciones que definen el comportamiento de los objetos creados a partir de la clase

Instanciación

Proceso de crear un objeto específico usando la palabra clave `new`

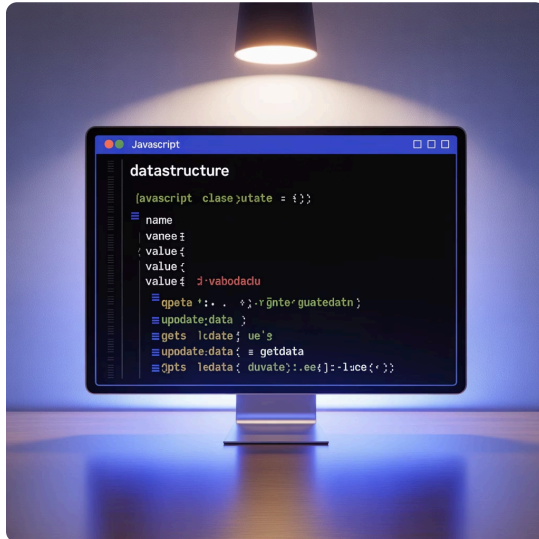
```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  
  saludar() {  
    console.log(`Hola, me llamo ${this.nombre}`);  
  }  
}
```

```
let p1 = new Persona("Luis", 30);  
p1.saludar(); // Hola, me llamo Luis
```

Propiedades y Métodos en Profundidad

Diferencias Clave

- **Propiedades:** Variables que almacenan el estado del objeto
- **Métodos:** Funciones que definen las acciones que puede realizar el objeto
- Ambos se definen dentro de la clase usando `this`



```
class Coche {  
  constructor(marca, modelo) {  
    this.marca = marca; // propiedad  
    this.modelo = modelo; // propiedad  
    this.encendido = false; // estado inicial  
  }  
  
  arrancar() { // método  
    this.encendido = true;  
    console.log("El coche está arrancando...");  
  }  
  
  obtenerInfo() { // método  
    return `${this.marca} ${this.modelo}`;  
  }  
}
```

Las propiedades mantienen el estado interno del objeto, mientras que los métodos proporcionan funcionalidad y pueden modificar estas propiedades. Esta separación es fundamental para mantener un código organizado.

Desestructuración: Extracción Inteligente de Datos

La desestructuración permite extraer valores específicos de objetos y arrays de manera concisa y elegante. Esta técnica reduce el código repetitivo y mejora la legibilidad.



Desestructuración de Objetos

```
let persona = { nombre: "Ana", edad: 25, ciudad: "Madrid" };  
let { nombre, edad } = persona;
```

```
console.log(nombre); // Ana  
console.log(edad); // 25
```



Desestructuración de Arrays

```
let numeros = [10, 20, 30, 40];  
let [a, b, ...resto] = numeros;
```

```
console.log(a); // 10  
console.log(b); // 20  
console.log(resto); // [30, 40]
```

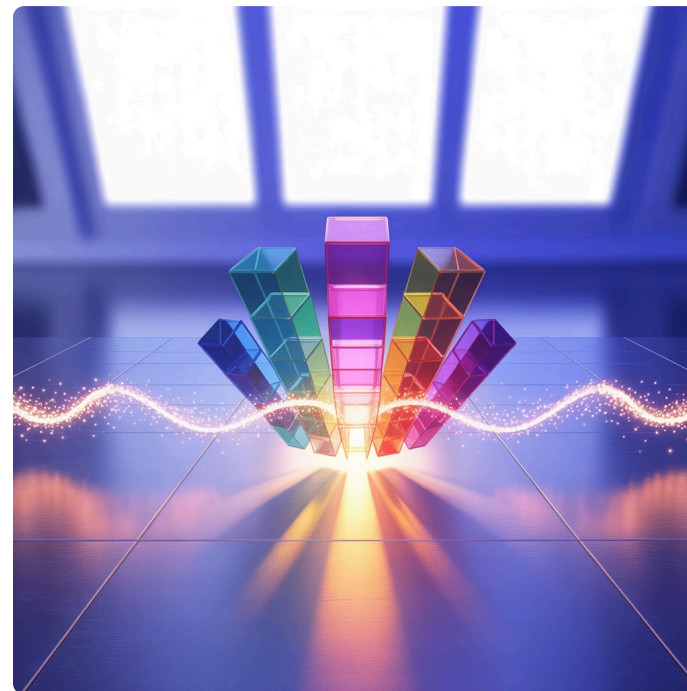
Operador Spread (...): Expandiendo Posibilidades

¿Qué hace el Operador Spread?

El operador spread (...) expande elementos de arrays u objetos en contextos donde se esperan múltiples elementos. Es perfecto para copiar, combinar y expandir estructuras de datos.

Casos de Uso Comunes

- Copiar arrays y objetos
- Combinar múltiples estructuras
- Pasar elementos como argumentos
- Crear copias inmutables



01

Con Arrays

```
let arr1 = [1, 2, 3];  
let arr2 = [...arr1, 4, 5];  
console.log(arr2); // [1, 2, 3, 4, 5]
```

02

Con Objetos

```
let obj1 = { nombre: "Ana" };  
let obj2 = { ...obj1, edad: 25 };  
console.log(obj2); // { nombre: "Ana", edad: 25 }
```

Operador Rest (...): Agrupando Elementos

Aunque usa la misma sintaxis que spread, el operador rest tiene el comportamiento opuesto: agrupa múltiples elementos en un array. Es especialmente útil en funciones con número variable de parámetros.



Parámetros Rest

```
function sumar(...numeros) {  
  return numeros.reduce((a, b) => a + b, 0);  
}  
  
console.log(sumar(1, 2, 3, 4)); // 10  
console.log(sumar(5, 10));    // 15
```



Desestructuración Rest

```
let [primero, segundo, ...resto] = [1, 2, 3, 4, 5];  
  
console.log(primero); // 1  
console.log(segundo); // 2  
console.log(resto);   // [3, 4, 5]
```

Spread vs Rest: Misma Sintaxis, Diferentes Propósitos

Operador Spread

Expande elementos de arrays/objetos

- Se usa en el lado derecho
- Descompone estructuras
- Crea copias y combina



Operador Rest

Agrupar elementos en arrays

- Se usa en el lado izquierdo
- Recolecta elementos
- Parámetros variables



Regla Mnemotécnica: Spread *esparce* (spread out), Rest *recoge* el resto de elementos. El contexto determina cuál operador estás utilizando.

Ejercicio Práctico Integrador

Consigna

Pondremos en práctica todos los conceptos aprendidos creando una clase `Estudiante` completa con desestructuración y operadores `spread`.

01

Crear la Clase

Define una clase `Estudiante` con propiedades `nombre`, `edad` y `notas` (array de números)

02

Método de Cálculo

Agrega un método `promedio()` que calcule el promedio de las notas usando `reduce()`

03

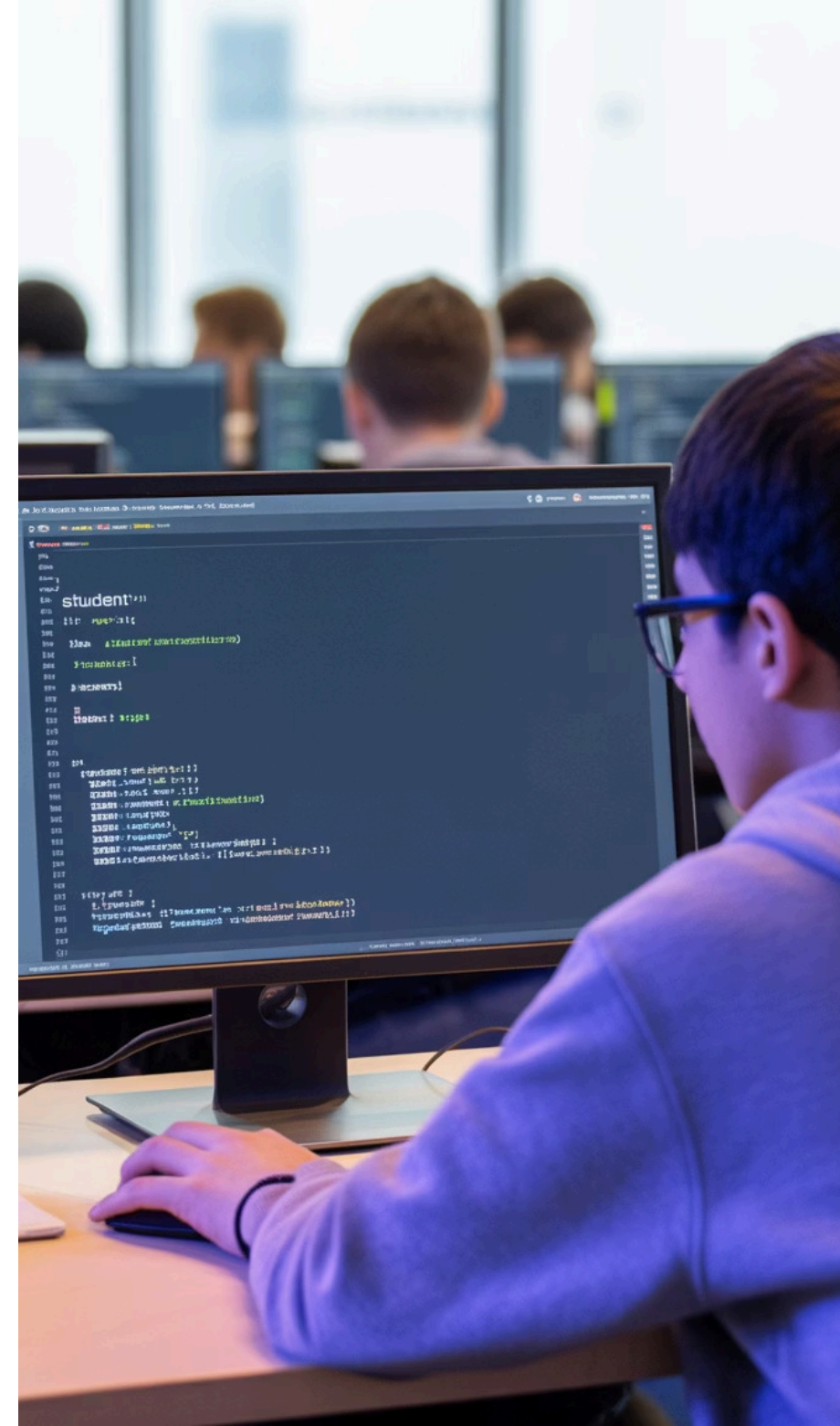
Desestructuración

Usa desestructuración para extraer solo el nombre de un estudiante

04

Operador Spread

Crea un nuevo objeto copiando el anterior pero cambiando la edad



Solución y Siguietes Pasos

Solución Completa

```
class Estudiante {  
  constructor(nombre, edad, notas) {  
    this.nombre = nombre;  
    this.edad = edad;  
    this.notas = notas;  
  }  
  
  promedio() {  
    return this.notas.reduce((a, b) => a + b, 0) / this.notas.length;  
  }  
}  
  
// Crear un estudiante  
let e1 = new Estudiante("María", 20, [8, 9, 10]);  
  
// Usar el método promedio  
console.log(e1.promedio()); // 9  
  
// Desestructuración  
let { nombre } = e1;  
console.log(nombre); // María  
  
// Spread para copiar y modificar  
let e2 = { ...e1, edad: 21 };  
console.log(e2);
```